

Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

Fecha de entrega: 3 de septiembre de 2026

Índice

1	Equipo	2
2	Repositorio	2
3	Dominio	2
4	Construcciones	3
5	Casos de prueba	3
6	Ejemplos	4

1. Equipo

Nombre	Apellido	Legajo	E-mail
Sol	Sommer	64.887	ssommer@itba.edu.ar
Sofía	Manzitti	65.521	smanzitti@itba.edu.ar
Morena	Díaz Macchi	65.558	mdiazmacchi@itba.edu.ar
Lucas Ariel	Crosta	67.266	lcrosta@itba.edu.ar

2. Repositorio

La solución será versionada en: <https://github.com/sofimanzitti/Trabajo-Especial-TLA>.

3. Dominio

Toda persona que cocina con cierta regularidad, desde quien organiza una comida familiar hasta un emprendimiento gastronómico pequeño, mantiene un repertorio de recetas escritas para una cantidad fija de porciones. Cada vez que debe cocinar para una cantidad distinta de comensales, o combinar varias recetas dentro de un mismo evento, se ve obligada a recalcular manualmente todas las cantidades, unificar los ingredientes que se repiten entre recetas, y reconstruir desde cero la lista de compras resultante.

Este mecanismo manual es factible para una o dos recetas, pero se degrada rápidamente a medida que crece la cantidad de recetas involucradas y la frecuencia de los eventos: el recálculo es repetitivo, propenso a errores aritméticos, y debe repetirse íntegramente ante cualquier modificación del menú. Más aún, el repertorio queda atrapado en un formato no estructurado (una libreta, un documento de texto), sobre el cual no es posible automatizar ninguna operación.

En este instante surge una **necesidad arquitectural**: se requiere poder especificar recetas y menús de forma programática, y que dicha especificación derive automáticamente las cantidades escaladas y la lista de compras consolidada. Debido a que la necesidad implica un escalamiento (un mismo repertorio de recetas debe servir para cualquier cantidad de comensales y cualquier combinación de menú), la solución es programática, y el dominio es específico (recetas, ingredientes, unidades de medida y menús), la construcción de un lenguaje y su compilador deberían solventar la necesidad.

Para reducir la complejidad del proyecto, una receta se modela como un conjunto de *ingredientes* con una cantidad requerida cada uno, junto con una secuencia de *pasos* de preparación con su duración. El lenguaje no representa el proceso físico de cocinar (temperaturas, técnicas, utensilios), sino únicamente la información necesaria para escalar cantidades, sustituir ingredientes faltantes por otros equivalentes, y calcular qué y cuánto hay que comprar.

Los artefactos de salida del compilador son dos: un archivo **CSV** con la lista de compras consolidada del menú, apta para ser abierta en una planilla de cálculo o impresa antes de ir al supermercado; y un documento **HTML** con las recetas ya escaladas y sus pasos de preparación, que oficia de hoja de ruta durante la cocción. Nótese que el programa de entrada describe únicamente el menú deseado, mientras que el compilador se ocupa del recálculo de las cantidades, de la resolución de las sustituciones y de sus conversiones de unidad, y de la consolidación de los ingredientes repetidos entre recetas.

La implementación satisfactoria de este lenguaje permitiría escribir cada receta una única vez y reutilizarla para cualquier cantidad de comensales o combinación de menú, versionando dicho repertorio con las mismas herramientas que se aplican al desarrollo de software, y eliminando por completo el recálculo manual.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (1). Se podrán declarar uno o varios *ingredients*, cada uno con un nombre, una unidad de medida (**grams**, **milliliters** o **units**) y, opcionalmente, una *density* expresada en g/ml, que permita convertir sus cantidades entre masa y volumen.
- (2). Se podrán crear una o varias *recipes*, y cada una tendrá nombre y una cantidad base de *servings* (porciones).
- (3). Cada *recipe* podrá requerir (**requires**) una o varias *quantities* de *ingredients* previamente declarados.
- (4). Cada *recipe* podrá definir uno o varios *steps*, cada uno con una descripción textual y una duración en *minutes*.
- (5). Las variables podrán ser de tipo *ingredient*, *quantity*, *recipe*, *menu*, *integer*, *boolean* o *string*. El resultado de un **scale** podrá asignarse a una variable de tipo *recipe* mediante el operador =, para poder referenciar la versión escalada más adelante.
- (6). Las variables podrán ser vectores de alguno de los tipos anteriores.
- (7). Se proveerán operadores relacionales como $<$, $>$, $=$, $\neq$, $\leq$ y $\geq$.
- (8). Se proveerán operaciones aritméticas básicas como $+$, $-$, $*$ y $/$.
- (9). Se proveerán operaciones lógicas básicas como **AND**, **OR** y **NOT**.
- (10). Se proveerán estructuras de control básicas de tipo **IF-THEN-ELSE** y **FOR**.
- (11). Se podrán definir reglas de **substitute** entre dos *ingredients*, indicando un *ratio* de equivalencia culinaria que multiplica la cantidad original al reemplazar uno por el otro. Si ambos *ingredients* no comparten la misma unidad de medida, la conversión se resuelve a partir de su *density*.
- (12). Se podrá **scale** una *recipe* a una nueva cantidad de *servings*, recalculando automáticamente las *quantities* requeridas y produciendo una versión escalada sin modificar la *recipe* original.
- (13). Se podrán agrupar una o varias *recipes* (cada una escalada a cierta cantidad de *servings*) dentro de un *menu*.
- (14). Se podrá **generate** una *shopping list* a partir de un *menu*, consolidando y sumando automáticamente las *quantities* de *ingredients* repetidos entre *recipes*. La consolidación es realizada internamente por el compilador/runtime; el lenguaje no expone al usuario la iteración ni las estructuras de datos necesarias para calcularla.

5. Casos de prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (1). Un programa que declare un *ingredient*.
- (2). Un programa que declare una *recipe* con 2 *ingredients* y 1 *step*.

- (3). Un programa que declare una *recipe* con múltiples *steps* secuenciales.
- (4). Un programa que escale (**scale**) una *recipe* a más *servings* que su cantidad base.
- (5). Un programa que escale una *recipe* a menos *servings* que su cantidad base.
- (6). Un programa que defina una regla de **substitute** entre dos *ingredients*.
- (7). Un programa que arme un *menu* con una sola *recipe*.
- (8). Un programa que arme un *menu* con varias *recipes* escaladas a distintas cantidades de *servings*.
- (9). Un programa que genere (**generate**) la *shopping list* de un *menu*, consolidando *ingredients* repetidos entre *recipes*.
- (10). Un programa que declare múltiples *ingredients* y múltiples *recipes* en un mismo archivo.
- (11). Un programa que sustituya un *ingredient* por otro declarado en una unidad de medida diferente, convirtiendo la cantidad mediante su *density*.

Además, los siguientes casos de prueba de **rechazo**:

- (1). Un programa malformado.
- (2). Un programa que declare una *recipe* que requiera un *ingredient* no declarado previamente.
- (3). Un programa que declare dos *ingredients* con el mismo nombre.
- (4). Un programa que intente escalar una *recipe* a una cantidad de *servings* igual o menor a cero.
- (5). Un programa que sustituya un *ingredient* por otro declarado en una unidad de medida diferente, sin que ninguno de los dos declare una *density* que permita resolver la conversión.
- (6). Un programa que defina una sustitución circular entre dos *ingredients* (A sustituye a B y B sustituye a A).

6. Ejemplos

En el Cód. (6.1), se puede ver la declaración de los ingredientes y de una receta simple con sus pasos de preparación, junto con el escalado de dicha receta al doble de porciones (lo cual duplica todas las *quantities* requeridas, sin alterar la receta original). El resultado se asigna a la variable **TortaGrande**, quedando disponible para su uso posterior.

```
// Declarar los ingredientes que se van a utilizar:
ingredient Harina in grams;
ingredient Manteca in grams;
ingredient Huevo in units;

// Declarar una receta con sus ingredientes y pasos:
recipe TortaDeManzana serves 8 {
  requires 300 Harina;
  requires 150 Manteca;
  requires 4 Huevo;

  step "Mezclar la harina con la manteca" takes 5 minutes;
  step "Agregar los huevos y batir" takes 3 minutes;
```

```
    step "Hornear la mezcla" takes 40 minutes;
}

// Escalar la receta anterior al doble de porciones:
TortaGrande = scale TortaDeManzana to 16 servings;
```

Listing 1: Declaración de ingredientes, de una receta simple y escalado de porciones asignado a una variable.

En el Cód. (6.2), que continúa el programa anterior (y por lo tanto reutiliza los *ingredients* *Harina* y *Manteca* ya declarados), se agrega un ingrediente y una segunda receta, se define una regla de sustitución, y se arma un menú combinando ambas recetas escaladas a distintas cantidades de porciones. Nótese que la sustitución cruza unidades de medida: el *ratio* 0.75 expresa la equivalencia culinaria entre ambas grasas, mientras que la *density* del aceite resuelve el pasaje de masa a volumen (150 g de manteca equivalen a 112,5 g de aceite, es decir, 122 ml). Finalmente se genera la lista de compras del menú mediante un único statement declarativo: el compilador recorre internamente cada receta y cada ingrediente del menú, y consolida las cantidades de forma automática. Como ambas recetas comparten *Harina* y *Manteca*, sus cantidades se consolidan en un único ítem de la lista.

```
// Declarar un ingrediente adicional y una segunda receta:
ingredient AceiteDeOlive in milliliters density 0.92;

recipe Empanadas serves 12 {
  requires 500 Harina;
  requires 100 Manteca;

  step "Preparar la masa" takes 15 minutes;
  step "Armar y hornear las empanadas" takes 25 minutes;
}

// Permitir reemplazar la manteca por aceite, en menor cantidad:
substitute Manteca with AceiteDeOlive ratio 0.75;

// Armar un menu combinando ambas recetas, cada una escalada:
menu Cumpleanos {
  include TortaDeManzana scaled to 20 servings;
  include Empanadas scaled to 24 servings;
}

// Generar la lista de compras consolidada del menu:
generate shopping list for Cumpleanos;
```

Listing 2: Sustitución con conversión de unidades, armado de un menú y generación declarativa de la lista de compras consolidada.